

ТЕСТОВ ПЛАН

**ПРОЕКТ - “DonateMate”
ИМЕ НА ОТБОРА - “PARADISE PALMS”**

ВЕРСИЯ НА ДОКУМЕНТА - 2.0

1. Въведение

DonateMate е софтуер, чиято цел е да помага на хора в нужда, като осигурява пространство за желаещи, които искат да дарят собствените си вещи и/или услуги. Целта на тестването на софтуера е да се гарантира, че основните бизнес процеси (регистрация, публикуване на нужди, сигурност) работят коректно преди официалното пускане на фаза “Closed Beta”. Тъй като приложението обработва лични данни (пр: телефони, локации), сигурността и валидацията са приоритет.

1.1 Цел 1

- Гарантиране на сигурността на потребителските профили чрез JWT (JSON Web Token);

1.2 Цел 2

- Валидиране на коректността на данните при създаване на заявки за дарения (Input Validation);

1.3 Цел 3

- Проверка на CRUD операциите (Create, Read, Update, Delete) за модул “Дарения”;

1.4 Цел 4

- Осигуряване на стабилност на API слой (Spring Boot REST Controllers).

2. Компоненти

Основните компоненти, които ще бъдат обект на тестване:

- **Модул за сигурност** (Authentication Module): Регистрация и Вход;
- **Модул за дарения** (Donation Module): Публикуване, редакция, изтриване и разглеждане на заявки;
- **Устойчивост на базата данни** (Database Persistence): Коректно записване на данни в H2 базата.

2.1. Оценка на риска

В таблицата са описани основните рискове, подредени от най-висок към най-нисък приоритет.

#	Риск	Impact	Предизвиква се от	Как може да се справим с него
1	Изтичане на лични данни / GDPR	Критичен	Endpoint на GET заявка е публичен и връща лични данни на потребители в чист вид.	Тестване на API-то през браузър без логин. Решението е скриването на телефона и имената на потребителите за гости (guests).
2	Редакция на чужди данни (IDOR)	Критичен	Липса на проверка дали ID-то на текущия потребител съпада.	Потребител А опитва да изтрие обява на потребител Б. При опит да се връща грешка 403.
3	Stored XSS (Cross Site Scripting)	Висок	Липса на изчистване на HTML тагове в полета при запис в базата данни.	Интегриране на HTML escaping на входа или изхода.
4	Неоторизиран достъп	Висок	Пробив в JWT логиката или неправилна конфигурация на security.	Проверка дали изтриването на обява изисква аутентикация.
5	Невалидни данни (Data Integrity)	Висок	Въвеждане на грешен формат на телефон и/или празни полета.	Тестове на Regex валидацията.
6	Загуба на данни (Persistence)	Среден	Използване на H2 (In-memory) база данни. При рестарт на сървъра данните изчезват.	При бъдещо развитие на проекта в режим продукция, проекта да бъде мигриран към друга база данни (пр: MSSQL, MySQL, etc.).
7	Performance при списъци	Нисък	Зареждане на голям брой от записи без странициране.	Тестване с голям обем от данни (пр: 1000+ записа) и добавяне на Pager.

3. Тестови сценарии

Тук са описани конкретните стъпки за ръчно тестване на приложението.

3.1. Тест на модул “Authentication”

- Като нов потребител трябва да мога да се регистрирам с валиден имейл и парола.
- Като регистриран потребител, при опит за вход с грешна парола, трябва да получа съобщение за грешка.
- Като регистриран потребител, при въвеждане на коректни данни, трябва успешно да вляза в системата.

Стъпка	Действие	Очакван резултат
1	Регистрация с валидни данни. Изпращане на заявка с валидни email, имена и парола (>6 символа).	Status 201 Created. Потребителят е създаден и се връща като валиден токен.
2	Регистрация със съществуващ email. Опит за повторна регистрация със същият email адрес.	Status 400 Bad Request. Грешка: “Email already in use.”.
3	Вход с грешна парола. Въвеждане на валиден email със сгрешена парола.	Status 401 Unauthorized. Системата отказва достъп.
4	Вход с валидни данни. Въвеждане на коректни email и парола.	Status 200 OK. Успешен вход и генериране на JWT токен.

3.2. Тест на модул “Donation Requests” (Създаване и Валидация) - 1/2

- Като не логнат потребител (guest), не трябва да имам право да публикувам обяви.
- При въвеждане на телефонен номер, съдържащ букви, системата трябва да ме спре с валидационна грешка.
- При попълване на всички задължителни полета коректно, обявата трябва да се визуализира в списъка.

Стъпка	Действие	Очакван резултат
1	Създаване на обява без токен. Опит за създаване на дарение без потребителят да е логнат в системата.	Status 401 Unauthorized. Достъпът е забранен.
2	Създаване на обява с невалиден. Опит за създаване на обява с телефон “PhoneNumber123”	Status 400 Bad Request. Грешка: “Email already in use.”
3	Създаване на обява с празни задължителни полета. Опит за създаване на дарение без да се запълват всички полета (или само space).	Status 400 Bad Request. Грешка за задължително поле.
4	Успешно създаване. Въвеждане на валидни данни (Локация, Телефон, Описание).	Status 201 Created. Обявата е записана успешно в базата данни.

3.3. Тест на модул “Donation Requests” (Разглеждане, Редакция и Изтриване) - 2/2

- Като потребител (логнат или гост), трябва да мога да видя списък с всички налични дарения, подредени по дата.
- Като собственик на обява, трябва да мога да редактирам обявата, в случай на допуснатата грешка.
- Като собственик на обява, трябва да мога да изтривам обяви от системата, ако вещта е предадена или услугата е изпълнена.
- При опит за изтриване на обява, която вече не съществува, системата трябва да върне съобщение за грешка.

Стъпка	Действие	Очакван резултат
1	Преглед на списък за всички обяви. Извикване на списъка с дарения без потребителят да е логнат в системата.	Status 200 OK. Системата връща списък с всички активни обяви, сортирани по дата.
2	Редактиране на обява. Изпращане на заявка за промяна на описание на съществуваща вече своя обява.	Status 200 OK. Полето се обновява с текущото време. Промяната е отразена в базата.
3	Изтриване на обява. Изпращане на заявка за изтриване на съществуваща вече своя обява.	Status 204 No Content / 200 OK. Обявата вече не се визуализира в общия списък.
4	Изтриване на несъществуваща обява. Опит за изтриване на обява с ID, което вече не съществува.	Status 404 Not Found. Връща се съобщение: “Donation request not found”.

3.4. Тест на сигурността

- Като злонамерен потребител, не трябва да мога да изтрия или редактирам обява, която не е създадена от мен (IDOR).
- Като не логнат посетител, не трябва да виждам личните телефонни номера или имейли на потребителите (GDPR).
- При опит за въвеждане на злонамерен код (XSS) в описанието, системата не трябва да го изпълнява.

Стъпка	Действие	Очакван резултат
1	Изтриване на чужда обява Опит за изтриване на обява (Потребител А), създадена от друг потребител (Потребител Б).	Status 403 Forbidden. Операцията е забранена.
2	Скриване на лични данни. Достъпване на списъка с дарения без логин (Public view).	Status 200 OK. Полетата за телефонен номер и имейл са скрити (заменени със звездички - *).
3	Stored XSS тест. Въвеждане на script (alert), в полето за описание на дарението.	Скриптът не трябва да се изпълнява при преглед на обявата от друг потребител.

4. Използвани термини

➤ **JWT (JSON Web Tokens):**

- Open standard (RFC 7519) for securely transmitting information between parties as a compact, URL-safe JSON object. It is widely used for authentication and authorization in web applications, APIs, and microservices;

➤ **API (Application Programming Interface):**

- Connection between computers or between computer programs. It is a type of software interface, offering a service to other pieces of software. A document or standard that describes how to build such a connection or interface is called an API specification. A computer system that meets this standard is said to implement or expose an API. The term API may refer either to the specification or to the implementation;

➤ **GDPR (General Data Protection Regulation):**

- An European Union regulation on information privacy in the European Union (EU) and the European Economic Area (EEA). The GDPR is an important component of EU privacy law and human rights law, in particular Article 8(1) of the Charter of Fundamental Rights of the European Union. It also governs the transfer of personal data outside the EU and EEA. The GDPR's goals are to enhance individuals' control and rights over their personal information and to simplify the regulations for international business;

➤ **IDOR (Insecure Direct Object Reference):**

- A type of access control vulnerability in digital security. This can occur when a web application or application programming interface uses an identifier for direct access to an object in an internal database, but does not check for access control or authentication;

➤ **XSS (Cross-Site Scripting):**

- A type of security vulnerability that can be found in some web applications. XSS attacks enable attackers to inject client-side scripts into web pages viewed by other users. A cross-site scripting vulnerability may be used by attackers to bypass access controls such as the same-origin policy. XSS effects vary in range from petty nuisance to significant security risk, depending on the sensitivity of the data handled by the vulnerable site and the nature of any security mitigation implemented by the site's owner network.